

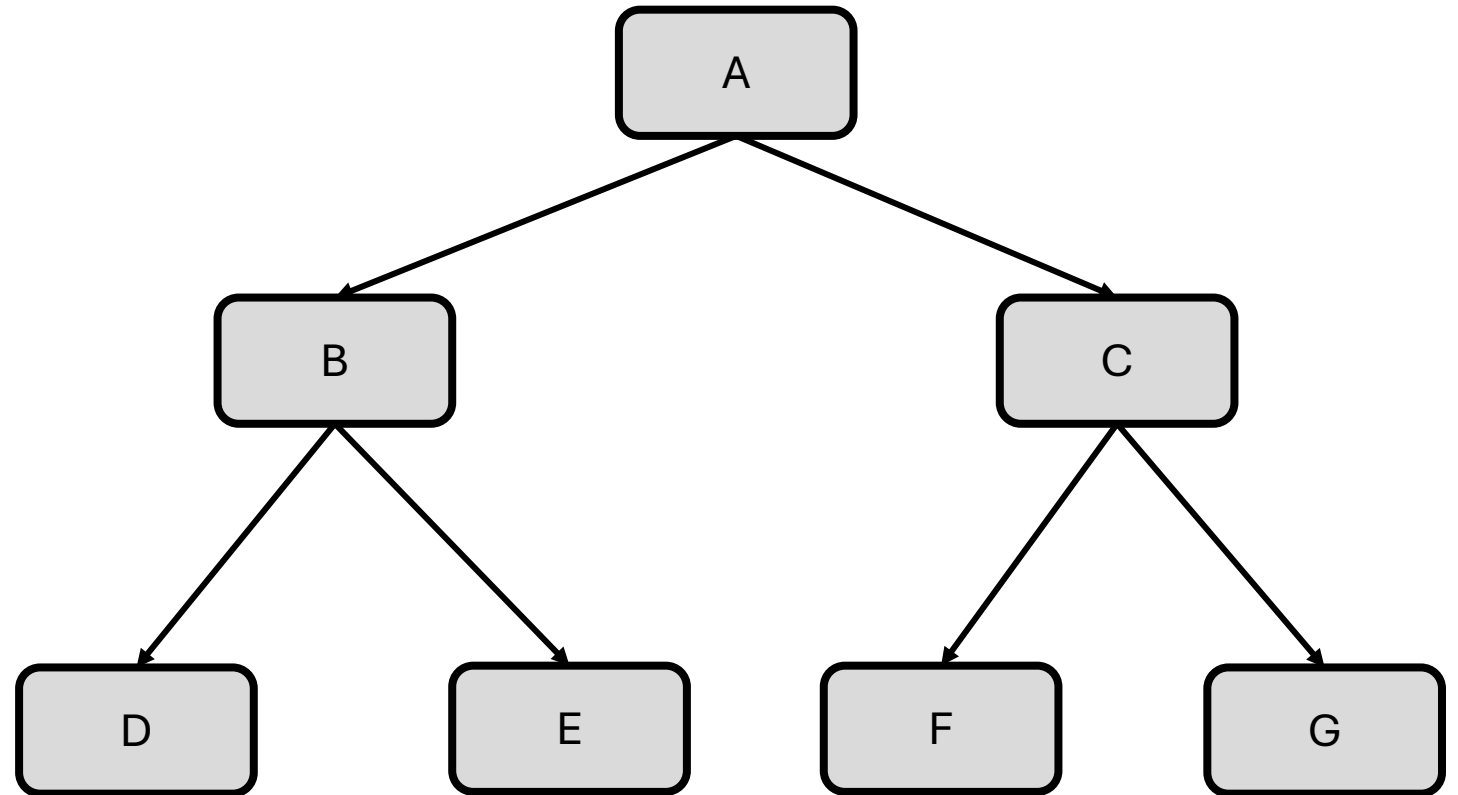
Decision Trees

Avi Mohan

Trees

- **Key Parts**

- Root
 - Parent
 - Child
 - Leaf
 - Subtree
 - Edge
-
- Depth
 - Height

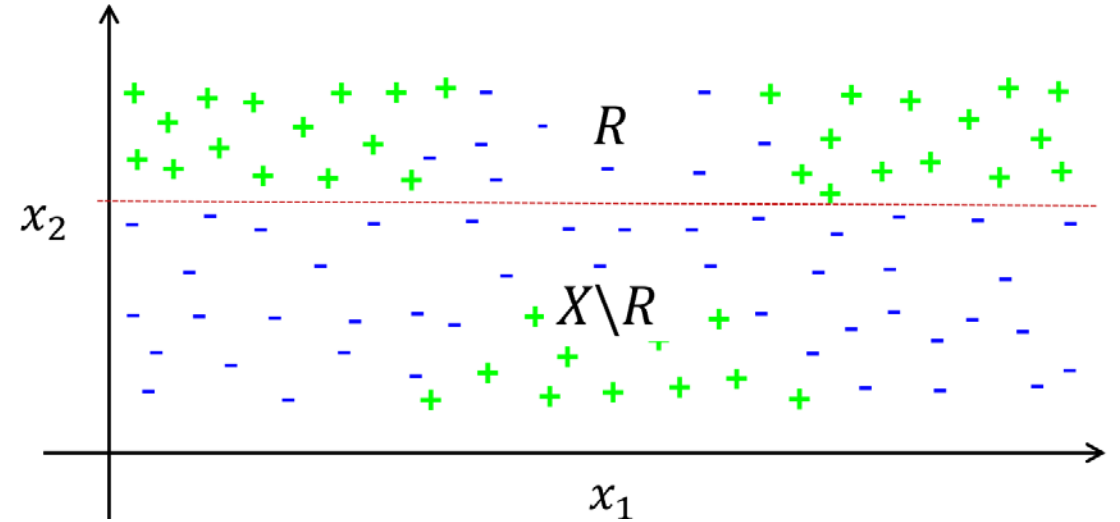


Decision Trees

- We use D_{train} to build a TREE STRUCTURE that **recursively** divides the feature space into regions with similar labels.
- **Underlying assumption:** data points of various classes are not randomly sprinkled across the feature space.
 - Appear in clusters of more or less **homogeneous** class assignments
- **Advantages:**
 - D_{train} need not be stored
 - Very fast during test time
 - (over kNN) No distance metric required

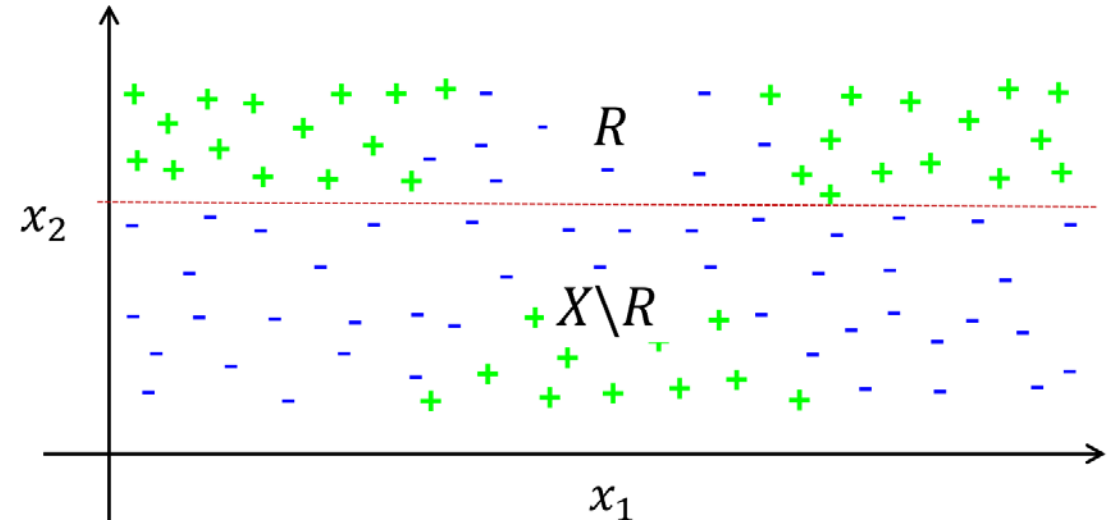
Splitting a node

- At each node we need to find
 - The feature to split on
 - Threshold for that feature
 - E.g. $\text{Latitude} < -23^\circ?$, or $\text{Time} > 9?$
- Loss function (impurity metric)
 - Region R
 - $\mathcal{C}$ classes (here, $\{0,1\}$)
 - $\widehat{p}_c$: proportion of training samples in R that are of class $\mathcal{C}$
- $L_{mc} = 1 - \max_c \widehat{p}_c$
 - “Purer” regions have lower loss



Splitting a node

- $L_{mc} = 1 - \max_c \widehat{p}_c$
 - “Purer” regions have lower loss
- R_p the parent region (subtree)
 - Splits into R_1, R_2
 - Child regions
- Split on the $(feature, threshold)$ pair that minimizes loss



$$j^*, t^* = \operatorname{argmax}_{j, t} L(R_p) - [\gamma_1 L(R_1) + \gamma_2 L(R_2)]$$

Splitting a node

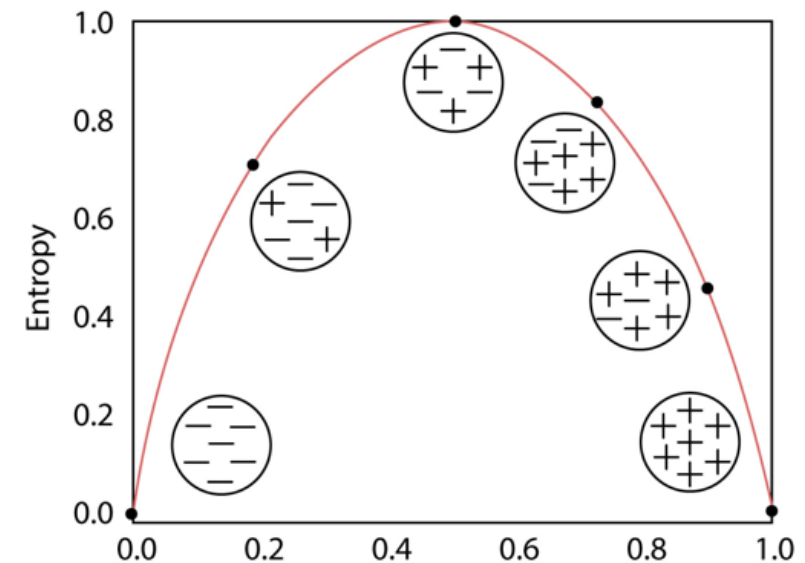
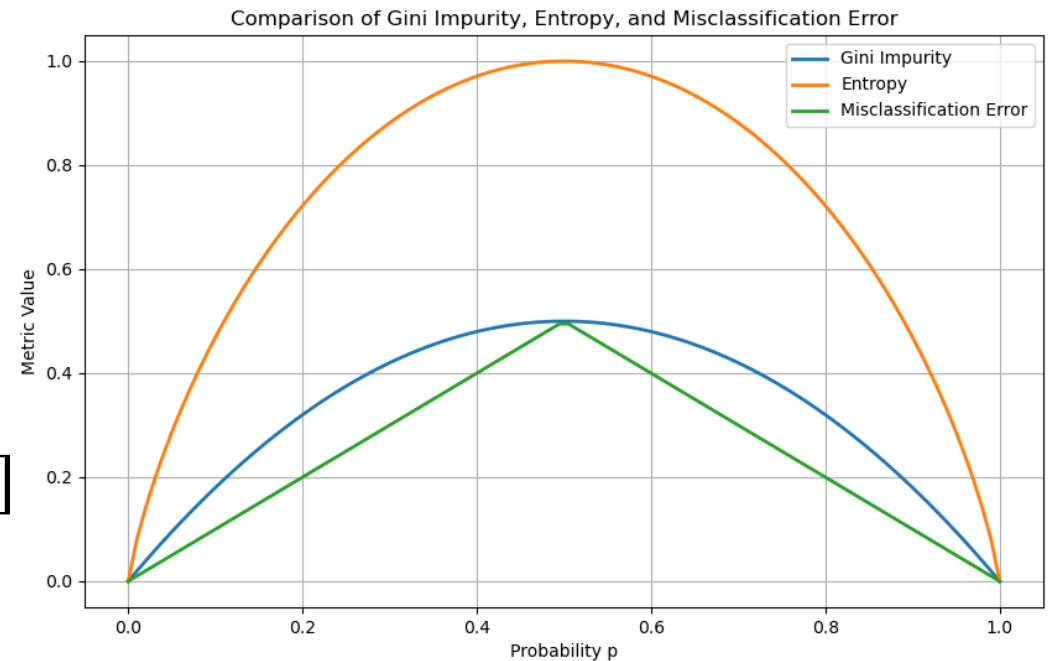
- Split on the $(feature, threshold)$ pair that minimizes loss
- $j^*, t^* = \operatorname{argmax}_{j, t} L(R_p) - [\gamma_1 L(R_1) + \gamma_2 L(R_2)]$

- Gini Impurity Index:

$$L_{gini}(R) = \sum_{c=1}^C \hat{p}_c (1 - \hat{p}_c)$$

- Entropy:

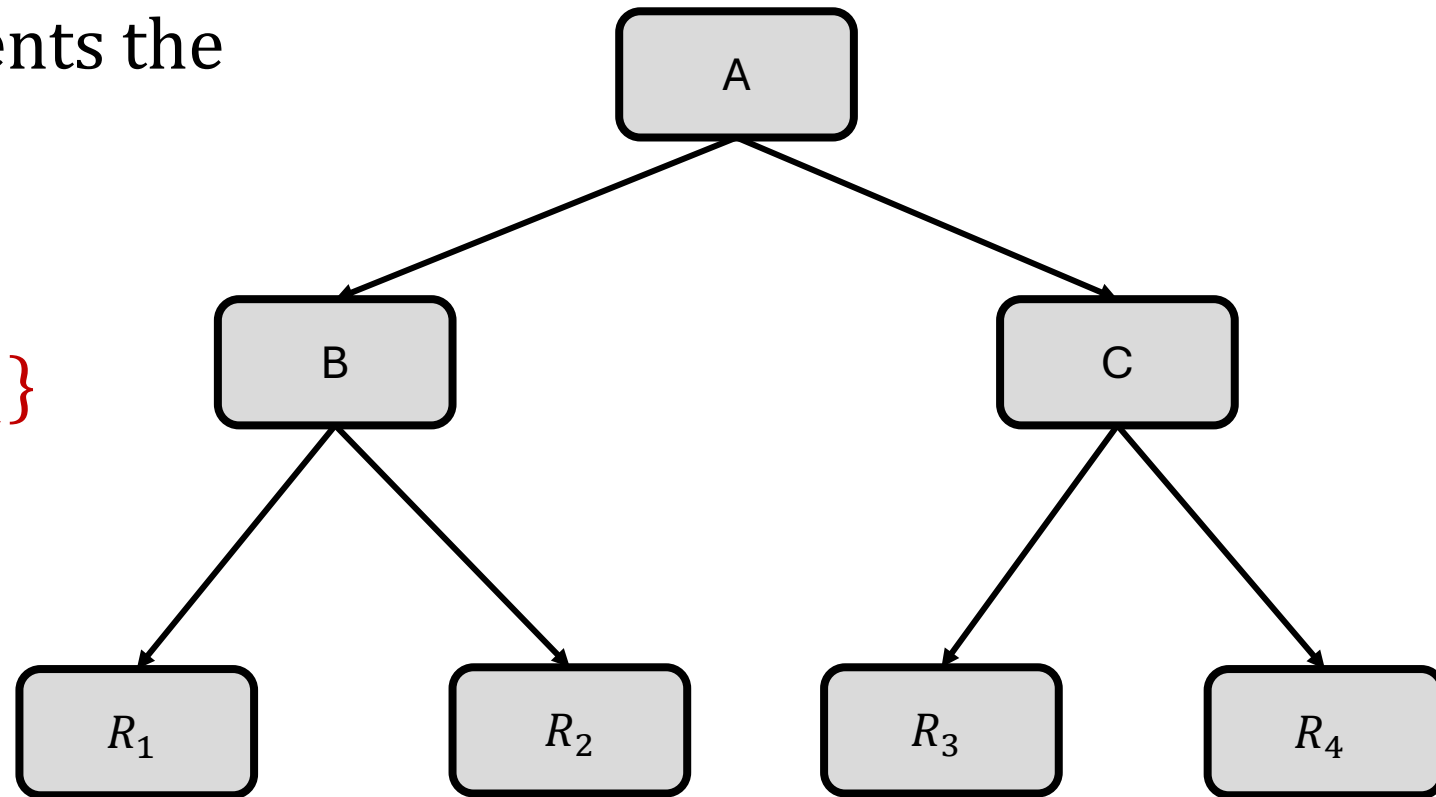
$$L_{ent}(R) = - \sum_{c=1}^C \hat{p}_c \log_2 \hat{p}_c$$



The function being implemented

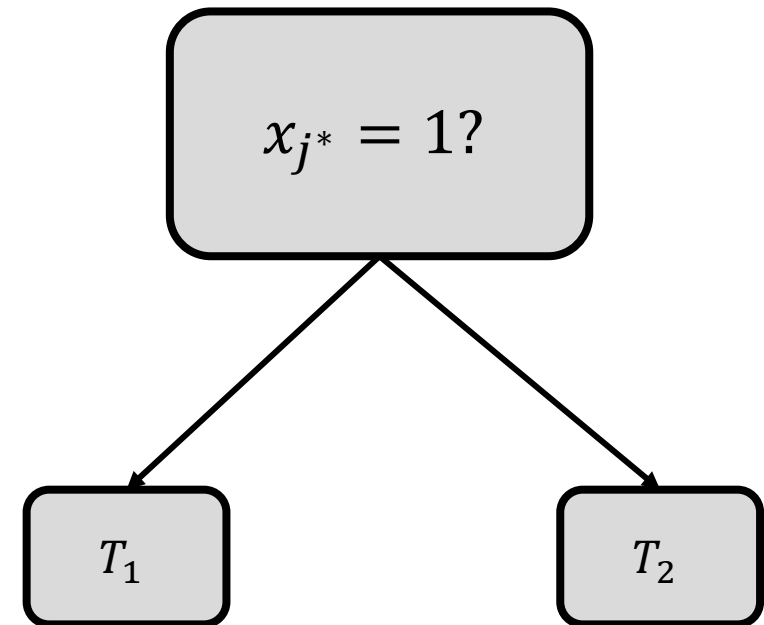
- A tree with T leaves implements the following function

$$f(\mathbf{x}) = \sum_{i=1}^T \hat{y}_i \mathbb{I}\{\mathbf{x} \in R_i\}$$



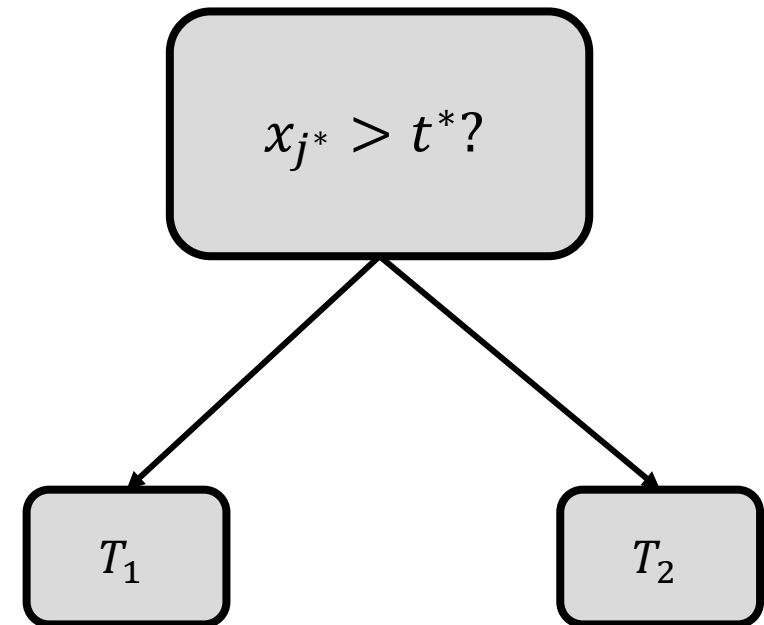
$ID3(S, A)$

- **INPUT:** $S \subset \mathcal{D}_{train}$, set of features $A \subset \{1, 2, \dots, K\}$
- **Terminating conditions:**
 - If all examples in S are labeled 0 or 1, return leaf 0 or 1 respectively
 - If $A = \emptyset$, return leaf whose value = majority of labels in S
- **Else:**
 - $j^* = \operatorname{argmax}_{j \in A} L(R_p) - [\gamma_1 L(R_1) + \gamma_2 L(R_2)]$
 - If all datapoints in S have the same x_{j^*}
 - Return leaf with majority label
 - **Else:**
 - $A' \leftarrow A \setminus \{j^*\}$
 - Return the tree with $T_1 = ID3(S_1, A')$, and $T_2 = ID3(S_2, A')$

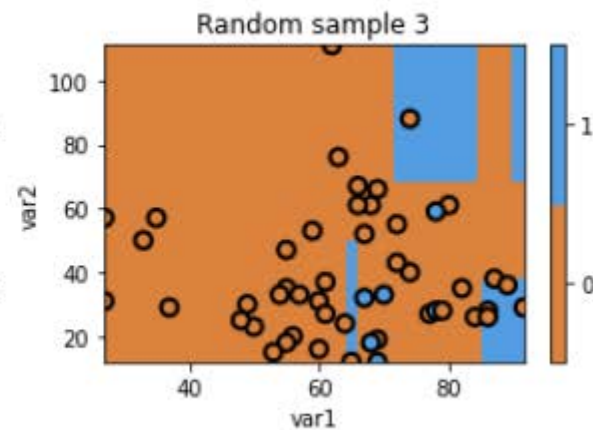
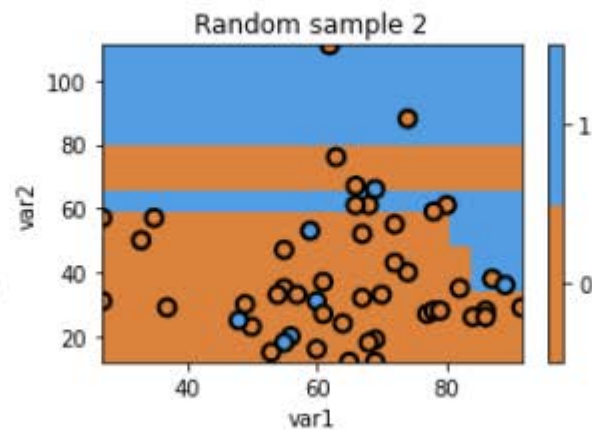
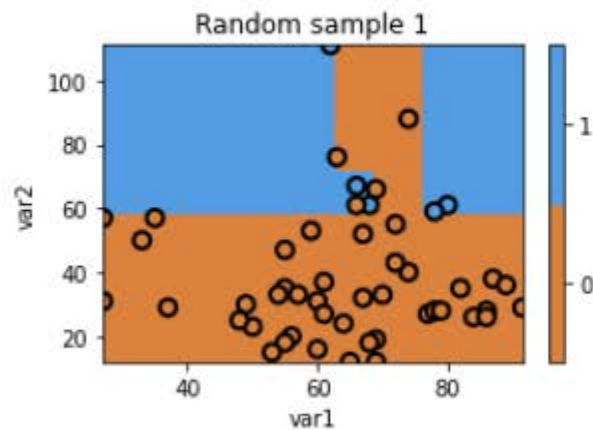
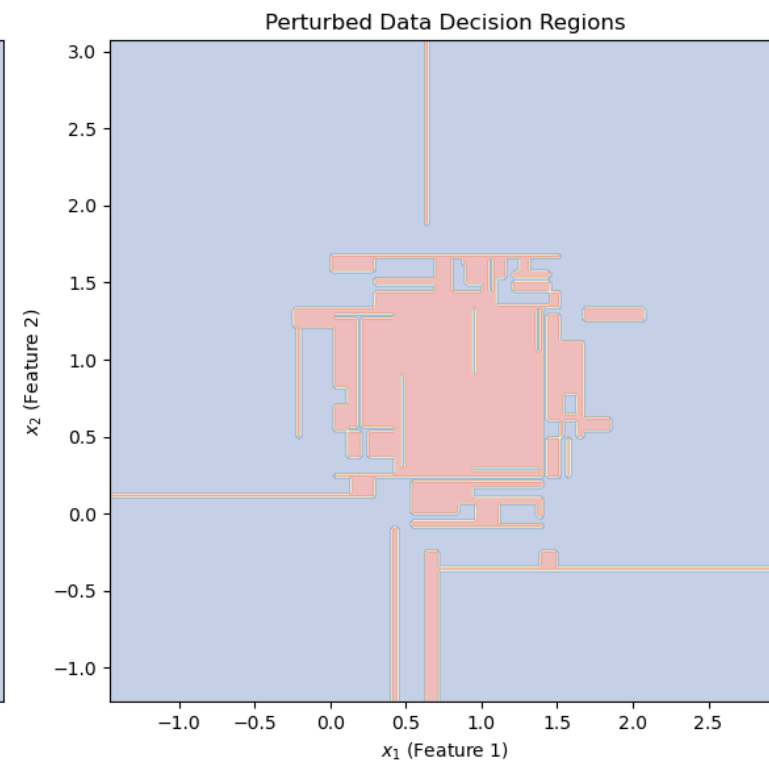
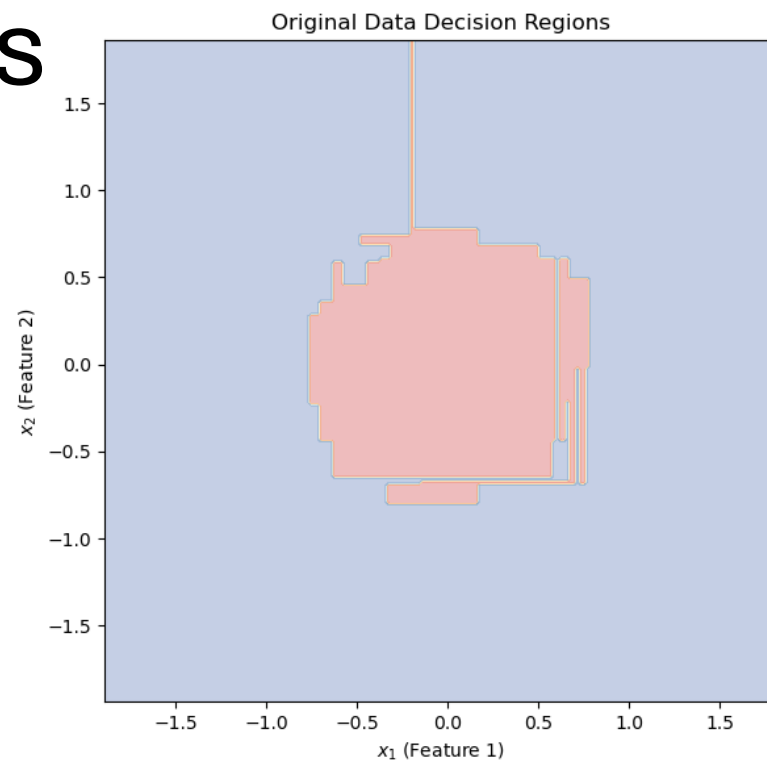


C4.5 in essence

- **INPUT:** $S \subset \mathcal{D}_{train}$, set of features $A \subset \{1, 2, \dots, K\}$
- **Terminating conditions:**
 - If all examples in S are labeled 0 or 1, return leaf 0 or 1 respectively
 - If $A = \emptyset$, return leaf whose value = majority of labels in S
- **Else:**
 - $j^*, t^* = \operatorname{argmax}_{j, t} \textit{Gain Ratio}(S, A)$
 - If all datapoints in S have the same x_{j^*}
 - Return leaf with label
 - Else:
 - Return the tree with $T_1 = ID3(S_1, A_1)$, and $T_2 = ID3(S_2, A_2)$



Overfitting in Trees



Overfitting in Trees

- **max_depth**: limit the maximum depth of the tree
- **min_samples_leaf**: minimum number of samples required for a node to be a leaf node.
- **min_samples_split**: minimum number of samples required to split a node.
- Cost-complexity pruning:
$$R_{\alpha}(T) = R(T) + \alpha |T|$$
 - $|T|$ = number of leaves (terminal nodes) in the tree T
 - $R(T)$ = misclassification error

Regression Trees

- Predicted label

$$\hat{y}_j = \frac{1}{|R_j|} \sum_{i:(x_i, y_i) \in R_j} y_i$$

- where $|R_j|$ = number of training datapoints in region R_j

- Squared Loss for region R_j

$$L_{\text{squared}}(R_j) = \frac{1}{|R_j|} \sum_{i:(x_i, y_i) \in R_j} (y_i - \hat{y}_j)^2$$